Task 2

1. Introduction

This report outlines the design, implementation, and analysis of a multithreaded C++ program designed to calculate the frequency of words in a given text file. The program leverages POSIX threads to divide the computational workload across multiple cores, improving execution speed for large text files. The program splits the code into chunks of data and processes them in parallel and joins the results safely.

2. Component Analysis

Functions

- **Clean:** A helper function that sanitizes input by removing non-alphabetic characters and converting all letters to lowercase. This ensures accurate counting.

```
string clean(string& word) {
    string result = "";
    for (char &c: word) {
        if (isalpha(c)) {
            result += tolower(c);
        }
    }
    return result;
}
```

- **Count:** Executed by all the working threads. It iterates through its assigned lines, utilizes a stringstream to extract individual words, cleans them, and updates its local frequency map. Afterward, it locks the mutex, updates the global map, and unlocks it.

```
void *count(void *arg) {
    thread_data *d = (thread_data *) arg;
    //a local map per thread
    map<string, int> local_count;

    // Process only the assigned slice of the vector
    for (int i = d->start; i < d->end; i++) {
        stringstream ss((*d->data)[i]);
        string word;
        while (ss >> word) {
            //call the cleaning function
            word = clean([&]word);
            local_count[word]++;
        }
    }
    //critical segment
    pthread_mutex_lock(&mtx);
    for (auto const &[word :const string , count :const int ]: local_count) {
        //modify the global result by what each thread found
        global_counts[word] += count;
    }
    pthread_mutex_unlock(&mtx);
    pthread_exit( retval: NULL);
}
```

4. Execution Flow

1. **Initialization:** The program validates command-line arguments, extracting the target filename and the optional thread count (defaulting to 4).
2. **File I/O:** The entire file is read sequentially line-by-line into the data vector.

```
int main(int argc, char *argv[]) {
    if (argc > 1) {
        fstream file(argv[1]);
        if (!file.is_open()) {
            //checking if the file could not be open
            cout << "Error: Could not open file " << argv[1] << endl;
            return 1;
        }

        string dummy;
        vector<string> data;
        while (getline([&]file, [&]dummy)) {
            data.push_back(dummy);
        }
        file.close();

        if (data.size() == 0) {
            cout << "File is empty." << endl;
            return 0;
        }
        //accept thread count from user or default to 4 threads
        int num_threads = (argc > 2) ? atoi(argv[2]) : 4;
```

3. **Thread Creation & Dispatch:**

   The program calculates a base chunk size by dividing the total number of lines by the thread count. A time snippet is taking to measure the effect of splitting the load over different threads.

   - A loop initializes thread_data for each thread. To ensure that all the lines are processed, the last thread is explicitly assigned all remaining lines to account for integer division truncation.
   - Threads are created and their segments are passed in the form of the thread data structure.

```
struct thread_data {
    //vector containing lines of the file
    const vector<string> *data;
    //the start index
    int start;
    //the end segement
    int end;
};
```

```
pthread_t th[num_threads];
thread_data t_args[num_threads];
int chunk = data.size() / num_threads;
auto start :time_point<system_clock, system_clock::duration> = chrono::high_resolution_clock::now();
for (int i = 0; i < num_threads; i++) {
    //store the address of the vector in the struct
    t_args[i].data = &data;
    //the start index
    t_args[i].start = i * chunk;
    //if there are enough remaining lines then take them or else take till the end of the vector
    //ensures that if there are more lines the last thread will take the rest
    t_args[i].end = (i == num_threads - 1) ? data.size() : (i + 1) * chunk;
    pthread_create(&th[i], attr: NULL, count, &t_args[i]);
}
```

4. **Synchronization:** The main thread calls pthread_join() in a loop, pausing its own execution until all worker threads have completed their tasks.

```
for (int i = 0; i < num_threads; i++) {
    pthread_join(th[i], thread_return: NULL);
}
```

5. **Local Output:** The local output is printed showing the result of each thread.

```
cout << "***The Local count***\n";
for (auto const& [word :const string , count :const int ] : local_count) {
    //local count for each thread
    cout << word << ": " << count << "\n";
}
cout << "\n";
```

6. **Output:** The final aggregated word counts are printed to the console alongside the total execution time.

```
// Print final results
for (auto const& [word :const string , count :const int ] : global_counts) {
    cout << word << ": " << count << "\n";
}
auto stop :time_point<system_clock, system_clock::duration> = chrono::high_resolution_clock::now();
auto duration = chrono::duration_cast<chrono::milliseconds>( d: stop - start);
cout << "Execution time: " << duration.count() << " milliseconds\n";
```

5. Performance and Concurrency Assessment

The current implementation ensures high thread safety and performance. By isolating the bulk of the computational work (string parsing, cleaning, and local counting) to thread-local variables, the program avoids creating a race condition by ensuring that each thread bulk execution is separated.

The critical section protected by a mutex lock and unlock and is kept strictly limited to the final merging of dictionaries. This ensures that the parallelized portion of the program runs at maximum efficiency.

6. Conclusion

The code implements multithreaded words count from an input text file. And the results when trying with different number of threads show a huge difference as when run with 2 threads the execution time is 130 ms. When increasing the number of threads to 20 threads, the execution time is 43 ms.

7. Github

Repo link: **https://github.com/youssef1316/OS_coursework1**

Commit sha: f7c6bc5713527c2ccf2e4d80fe3572e916806121